Distilling Linearized Behavior into Non-Linear Fine-Tuning for Effective Task Arithmetic

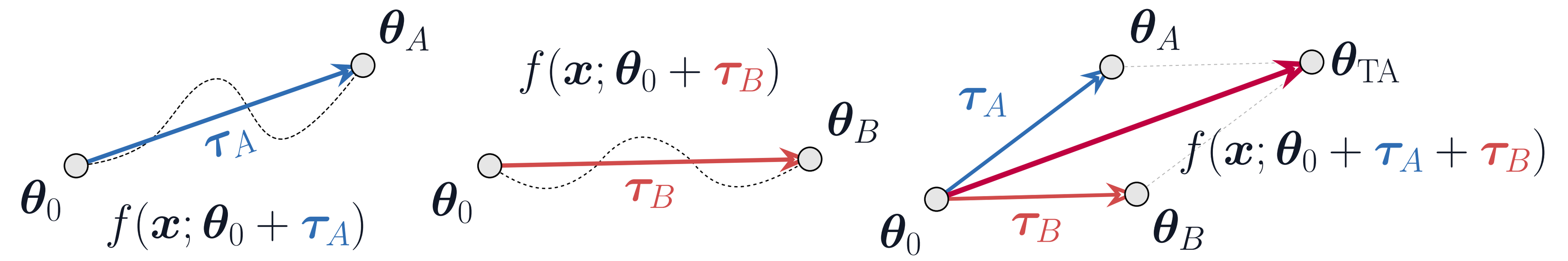
Thomas Sommariva[1], Francesca Morandi[1,2], Simone Calderara[1], Angelo Porrello[1]

[1]AImageLab, University of Modena and Reggio Emilia, Italy [2]University of Pisa, Italy

arXiv GitHub

Background: Task Arithmetic

Idea. Fine-tune models, then combine weights (task vectors $\boldsymbol{\tau} = \boldsymbol{\theta} - \boldsymbol{\theta}_0$) through addition (***multitasking***) or subtraction (***unlearning***).

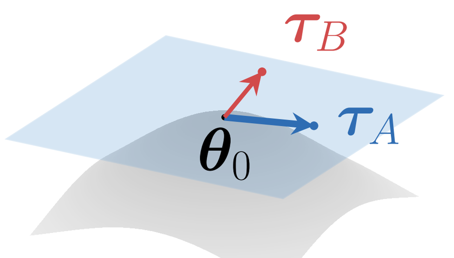


Weight disentanglement. Data from distinct regions in input space affect separate, non-overlapping regions of the activation space.

$$f(\boldsymbol{x}; \boldsymbol{\theta}_0 + \boldsymbol{\tau}_A + \boldsymbol{\tau}_B) = f(\boldsymbol{x}; \boldsymbol{\theta}_0)\,\mathbb{I}_{\boldsymbol{x}\notin A\cup B} + f(\boldsymbol{x}; \boldsymbol{\theta}_0 + \boldsymbol{\tau}_A)\,\mathbb{I}_{\boldsymbol{x}\in A} + f(\boldsymbol{x}; \boldsymbol{\theta}_0 + \boldsymbol{\tau}_B)\,\mathbb{I}_{\boldsymbol{x}\in B}$$

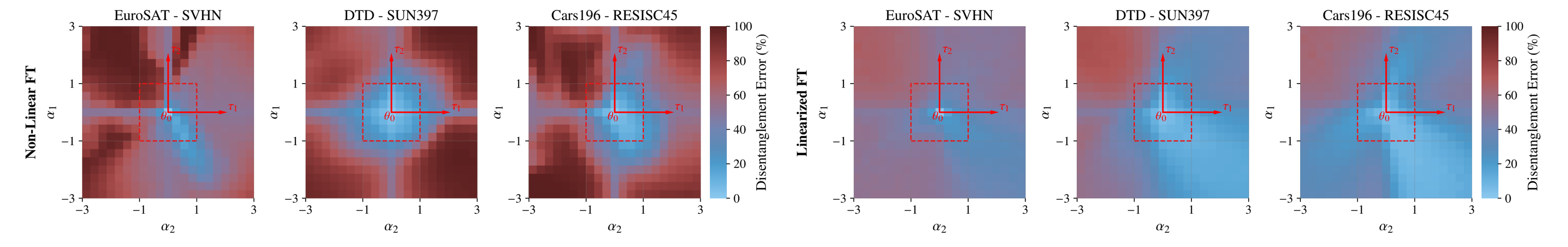
Prior works. Fine-tune the **linearized model**.

$$f_{\text{lin}}(\boldsymbol{x}; \boldsymbol{\theta}) = f(\boldsymbol{x}; \boldsymbol{\theta}_0) + \mathrm{J}_{\boldsymbol{\theta}} f(\boldsymbol{x}; \boldsymbol{\theta}_0)(\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$



Linearized models have shown stronger performance in Task Arithmetic.

Non-linear fine-tuning | **Linearized fine-tuning**



$$\xi(\alpha_1, \alpha_2) = \sum_{t=1}^{2} \mathbb{E}_{\boldsymbol{x}\sim\mu_t}\left[\text{dist}\left(f(\boldsymbol{x}; \boldsymbol{\theta}_0 + \alpha_t\boldsymbol{\tau}_t), f(\boldsymbol{x}; \boldsymbol{\theta}_0 + \alpha_1\boldsymbol{\tau}_1 + \alpha_2\boldsymbol{\tau}_2)\right)\right]$$

Reduced weight disentanglement error!

Linearized fine-tuning: Pros & Cons

Mathematically tractable
Exact characterization of task interactions

Weight Disentanglement
Reduced interference between task vectors

Robust task arithmetic
Better scaling and vector composition

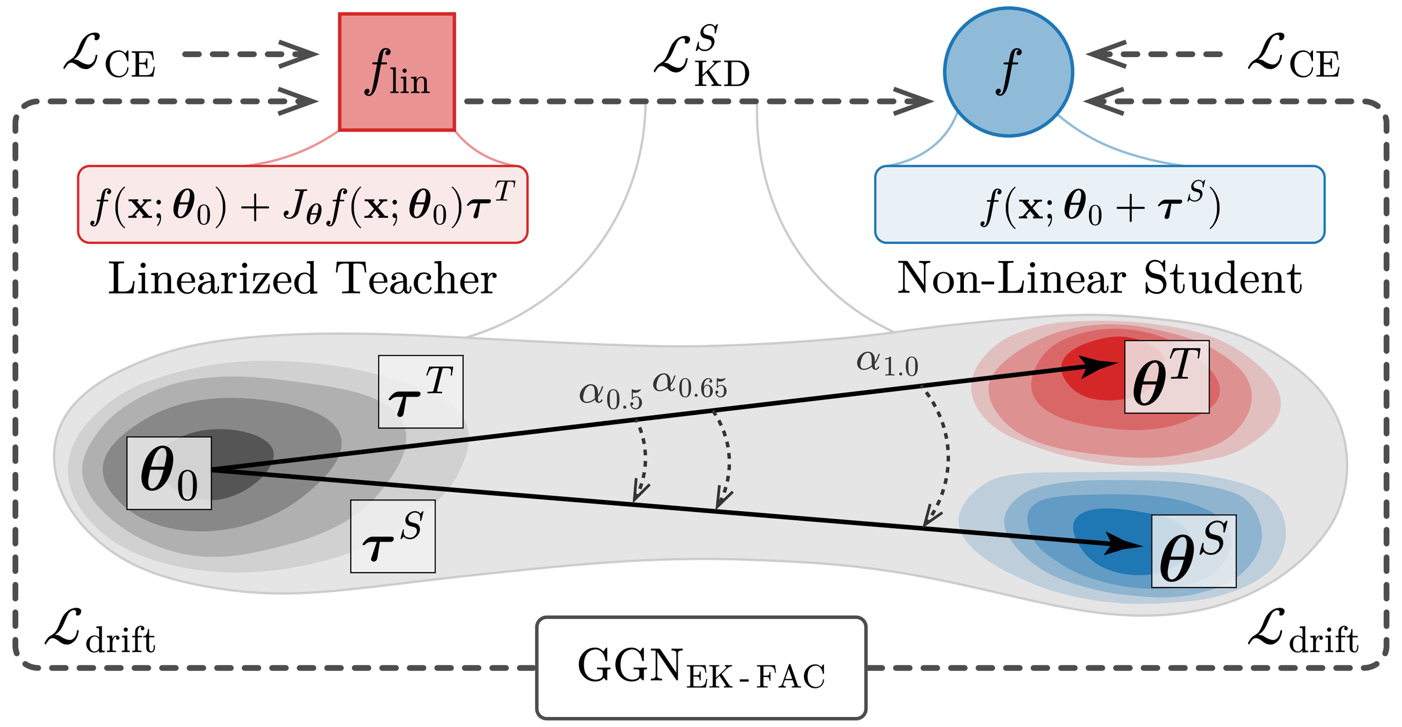
Computational cost
Double inference and training cost due to Jacobian–vector products

Reduced expressivity
Constrained to a local linear approximation

TL;DR: How to fine-tune for improved merging?

We distill a model trained via **linearized fine-tuning** (*teacher*) into a model trained via **conventional fine-tuning** (*student*), preserving the benefits of linearized models with no inference-time overhead and enhanced expressivity.

DELTA: DistillEd Linearized Task Arithmetic



DELTA: the student *i)* distills activations from a linearized teacher via the APKD loss *ii)* it optimizes a term that encourages weight disentanglement.

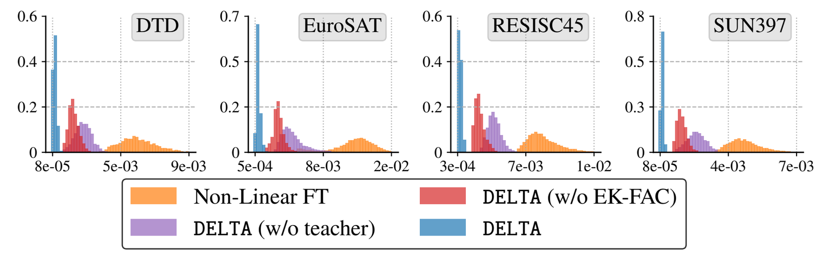
Along-Path Knowledge Distillation (APKD)

$$\mathcal{L}^S_{\text{KD}} = \left\| f\left(\boldsymbol{x}_i; \boldsymbol{\theta}_0 + \alpha\boldsymbol{\tau}^S_t\right) - f_{\text{lin}}\left(\boldsymbol{x}_i; \boldsymbol{\theta}_0 + \alpha\boldsymbol{\tau}^T_t\right) \right\|_2^2 \quad \boxed{\alpha \sim \mathcal{U}(0.5, 1)}$$

We distill the student along the linear path from the base model, ensuring that the linearized behavior is inherited at every scaling coefficient.

DELTA reduces the **linearization error**:

$$\xi_{\text{lin}}(\boldsymbol{x}) = \left\| \frac{1}{T}\sum_{t=1}^{T} f(\boldsymbol{x}; \boldsymbol{\theta}^S_t) - f\left(\boldsymbol{x}; \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{\theta}^S_t\right) \right\|_1$$



Weight Disentanglement via curvature regularization

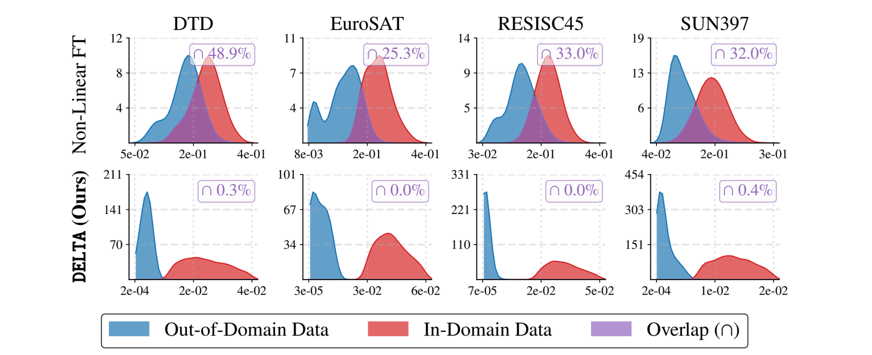
$$\mathcal{L}^t_{\text{drift}} = \boldsymbol{\tau}^{S\top}_t \text{GGN}_{\text{EK-FAC}} \boldsymbol{\tau}^S_t$$

We extend the approach in [1] with a curvature matrix estimated on a reference dataset.

[1] Porrello et al., "Dataless Weight Disentanglement in Task Arithmetic via Kronecker-Factored Approximate Curvature," ICLR 2026.

DELTA encourages **task-localized edits:**

$$d_t(\boldsymbol{x}) = \frac{1}{d}\left\| f(\boldsymbol{x}; \boldsymbol{\theta}^S_t) - f(\boldsymbol{x}; \boldsymbol{\theta}_0) \right\|_2^2$$



Experiments

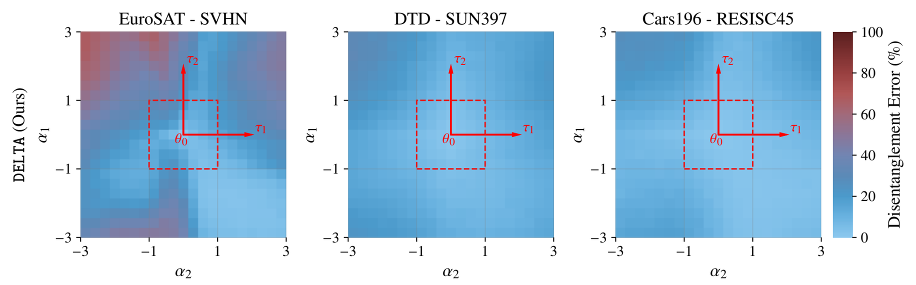
SOTA with Full FT

Method	8-Vision				14-Vision		6-NLI	
	ViT-B/32		ViT-L/14		ViT-B/32		T5-Base	
	Abs.	Norm.	Abs.	Norm.	Abs.	Norm.	Abs.	Norm.
Pre-trained	48.4	—	65.0	—	57.8	—	61.7	—
Individual	92.8	—	95.8	—	90.2	—	85.9	—
Linearized models								
Linear FT	78.9	89.8	88.0	94.8	76.7	87.0	76.4	93.5
$\boldsymbol{\tau}$Jp	85.6	98.2	91.1	98.5	85.4	**97.1**	**82.5**	**100**
TAK	86.1	97.8	91.6	99.3	84.7	96.0	79.5	98.8
Non-Linear models								
Non-Linear FT	73.5	80.4	84.5	89.7	68.9	76.1	78.2	91.6
TaLoS	77.9	87.7	84.7	91.1	74.9	84.4	76.7	89.8
Attn. Only FT	78.2	86.3	88.2	93.8	73.4	81.5	76.7	91.4
DELTA (ours)	**88.3**	**98.3**	**92.7**	**99.5**	**86.0**	96.0	82.4	95.5

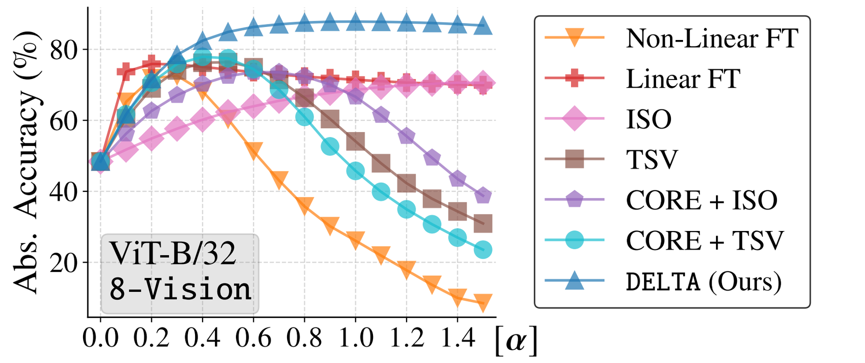
SOTA with LoRA FT

Method	8-Vision				14-Vision	
	ViT-B/32		ViT-L/14		ViT-B/32	
	Abs.	Norm.	Abs.	Norm.	Abs.	Norm.
Non-Linear FT	72.6	79.5	85.3	90.5	68.4	75.7
Linear FT	75.9	86.2	87.1	94.0	75.5	85.5
Iso-C	70.6	77.9	85.3	90.7	71.9	80.0
TSV-M	76.4	83.9	88.9	94.4	74.3	82.1
Core + Iso-C	73.6	81.2	87.4	93.0	72.3	80.0
Core + TSV-M	77.9	85.6	89.1	94.6	74.5	82.5
DELTA (ours)	**87.5**	**97.9**	**92.2**	**99.1**	**85.7**	**96.3**

Weight disentanglement!

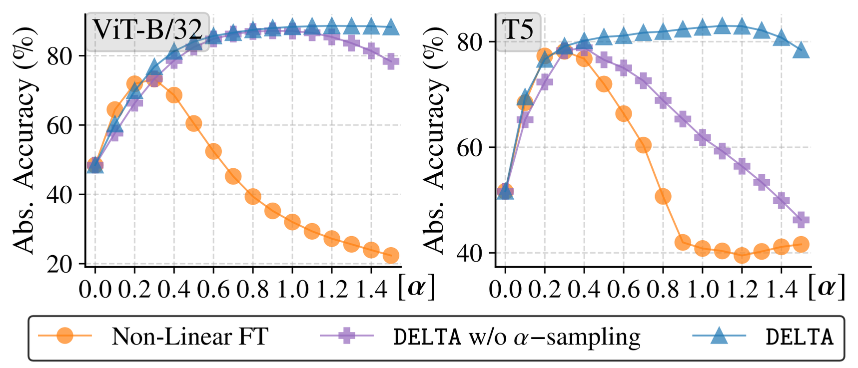


Better and more robust LoRA merging!

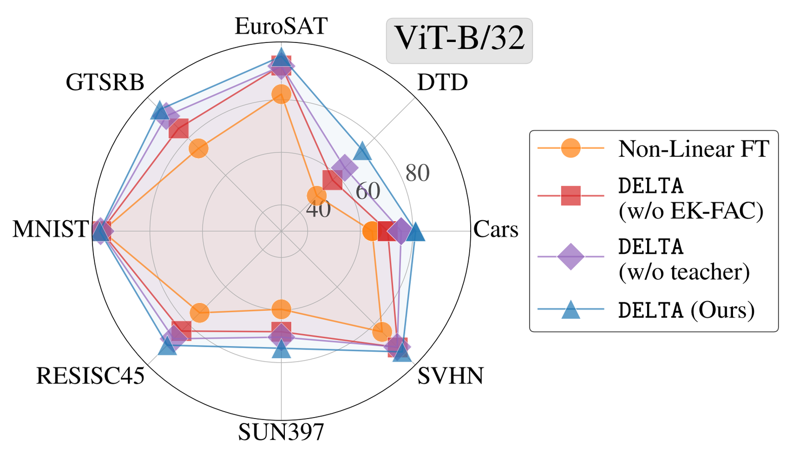


Finding. Linearity to weight perturbations, defined in *weight space*, can be transferred to standard non-linear models by matching in *activation space*.

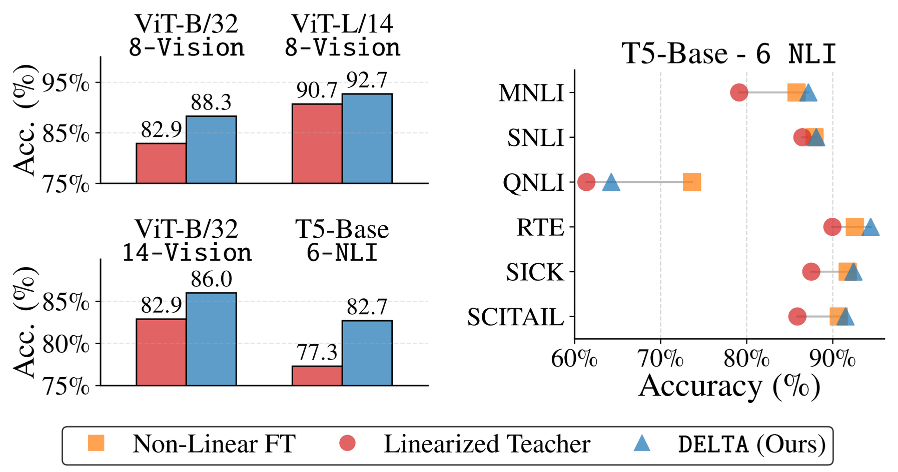
APKD improves α robustness



Ablation analyses



Preserved expressivity



Application to LLMs